

GCC Data Console

Project Description — a Node.js / React rebuild of the original Course Data Manager

Background

The Department of Information and Communications Technology (DICT), together with Globe Telecom and Google, runs the Google Career Certificates (GCC) program — free, job-ready training in tracks such as Data Analytics, IT Support, Project Management, and UX Design. Every week, Coursera publishes a CSV report of enrolled learners (name, email, specialization, enrollment and completion dates, and more), and DICT keeps a separate masterlist CSV mapping each learner's name and email to a region.

The original workflow required staff to reconcile these two files by hand in Excel — a slow, error-prone process once the reports grew to thousands of rows. The first version of this project (“Course Data Manager”) automated that reconciliation with a PHP/MySQL web app. This document describes the current version, **GCC Data Console**, which reimplements the same workflow on a Node.js/Express API, a React (Vite) frontend, and an embedded SQLite database.

What the System Does

A user uploads the two CSV files through the browser. The API validates the structure and contents of both files — column headers, required fields, date formats, allowed values — before writing anything to the database. If validation passes, the records are imported in a single transaction and merged with region data from the masterlist. The user can then browse, filter, and sort the merged records, review any duplicate accounts, and export the current view to CSV or Excel.

Specific Objectives

- Validate the structure and cell-level contents of both CSV files before import.
- Import validated CSV data into a persistent database as a single, atomic operation.
- Let the user filter and sort imported records by specialization, region, status, and dates.
- Export the currently filtered/sorted view to CSV or Excel.
- Flag learners whose name appears with more than one email address (duplicate accounts) and let the user export that list separately.
- Remove the SQL-injection and unescaped-output issues present in the original implementation, and correct a data-import bug that could silently drop failed rows.

System Architecture

The system is split into two independent pieces that communicate over a JSON REST API:

Layer	Technology	Responsibility
-------	------------	----------------

Frontend	React 18 + Vite	Import screen (drag-and-drop CSV upload, validation-error log, duplicate review modal) and Records screen (filter toolbar, sortable table, pagination, export buttons).
Backend	Node.js + Express	Parses and validates uploaded CSVs, writes to the database inside a transaction, and serves filtered/sorted/paginated record queries.
Database	SQLite (node:sqlite)	Single-file embedded database — no separate database server to install or configure.

Improvements Over the Original Implementation

Security

The original filter/sort screen built its SQL query by concatenating query-string values directly into the statement, which allowed SQL injection through the filter or sort parameters. The rebuilt API only ever builds queries from a fixed whitelist of column names and passes user values as bound parameters, so no value from the request can change the shape of the query. Record values are also rendered through React, which escapes output by default, closing an XSS path that existed when values were echoed directly into HTML.

Data Integrity

The original import routine tracked success with a boolean that a few code paths set backwards, so a failed insert could be reported as successful without the user knowing. The new import runs inside a single database transaction: either every row is written, or none are, and the API returns a structured error either way.

Validation

The original name-validation pattern only accepted letters and spaces, rejecting real names containing hyphens or apostrophes (e.g. “O’Brien”). The pattern now allows letters, spaces, hyphens, apostrophes, and periods, including accented characters.

Interface

The import screen now uses drag-and-drop upload zones for both files and shows validation problems as a scrollable, row-and-column-referenced log instead of a single block of text. The records screen adds click-to-sort columns, a filter toolbar, pagination, and one-click CSV or Excel export, replacing several overlapping stylesheets from the original with one shared set of styles.

Scope

The system accepts exactly two CSV inputs: the Coursera learner report (13 fixed columns) and the DICT masterlist (Region, Name, Email). Uploaded files are checked for the correct number and names of columns and for valid values in every required cell before any data is imported. Once imported, records can be filtered, sorted, paginated, and exported; learners whose name is associated with more than one

email are surfaced as possible duplicate accounts.

Limitations

- Only CSV files in the exact Coursera-export and masterlist formats described above are accepted; other layouts are rejected rather than auto-corrected.
- Invalid cells are reported to the user, not fixed automatically — the offending file must be corrected and re-uploaded.
- Duplicate accounts are flagged and exported for review; they are not merged or deleted automatically.
- The database is a single local SQLite file with no built-in multi-user access control — it is intended for a single operator running the tool locally, not a shared, always-on deployment.

Tools & Requirements

Development uses Visual Studio Code. The only runtime requirement is Node.js 18 or later — the SQLite database is created automatically on first run, so no separate database server (e.g. XAMPP/MySQL, as the original version required) needs to be installed. See the accompanying User's Manual for setup and day-to-day usage instructions.